23 Feb 2026 v1

COMP281 2025-2026 – Assignment 2

- In the following, you will find the tasks that constitute Assignment 2. They will be also available via Code Grade on the canvas module for COMP281.

- You need to write a C program (not C++ or C#) that solves each problem – it must read the input, as specified in the problem description then print the solution to the given problem for that input.
    - Note that code is "correct" only if it **correctly implements a solution to the problem stated** in the assignment, not "if CodeGrade accepts it".
    - That is, even if CodeGrade accepts your code, it could be wrong. Read the problems carefully.
    - Make certain that your coding is clear so that it may be easily understood by the assessor (i.e. use spaces and blank lines appropriately to improve legibility). The final mark for the assignment will be based on whether your programs pass Code Grade's auto tests, but also the assessor's judgement on the code you've submitted.

- Input is read from the standard input, in the same way that you read input from the keyboard as shown in lectures (e.g., using scanf). Output is also printed to the standard output, as you have seen (e.g., using printf).

- **Do not include any additional prompt messages in your programs**. The appearance of such text in the output from your code will mean that it cannot possibly match the output that Code Grade is checking for. N.B. For the week 2 assignment, this was not the case (CodeGrade was set to ignore additional outputs. That feature has been switched off for this assignment).

- This assignment is worth 50% of the total mark for COMP281.
    - Both tasks are weighted equally.
    - For each task, your solution can earn a total of 50 points (scaled from 20 points per solution in assignment 1).
        - 25 points for "Functionality and Correctness" awarded for programs that correctly solve the problem for all test cases.
        - 25 points for "Programming style, use of comments, indentation and identifiers" awarded depending on the style, comments and efficiency of the solution
    - The final grade results from normalising the earned points to a scale of 100.
    - Please note, you must submit at least four different versions of each of your solutions to CodeGrade, which when taken together show the assessor how your code has evolved. This should be more than just minor differences such as changing the name of a variable or adding a comment. You may of course do your development work on whatever platform you wish, but making interim submissions to check whether your approach works on CodeGrade is recommended anyway. The penalty for failing to do this is 15 marks per problem (i.e. if you submit only once for task 1 and once for task 2, you will lose 30 marks).
    - See separate "comp281-detailed-marking-guidelines for A2.pdf" for more details of how the marks are allocated.

Submission Instructions

- Submit your solution to each part of the assignment via Canvas/CodeGrade. Make **at least four** such submissions as your code is being developed, showing more than just very minor differences.
- The deadline for this assignment submission is **Friday 6-Mar-2026 at 23:59**
- Penalties for late submission apply in accordance with departmental policy as set out in the student handbook, which can be found at: http://intranet.csc.liv.ac.uk/student/ug-handbook.pdf

Task 1.

Title: Run Length Encoding of ASCII Art

Description

In the very early days of computing, images were produced using ASCII characters. Such ASCII Art was very popular and, since storage was limited, simple techniques were developed to compress such images. One such simple technique of lossless compression was Run Length Encoding. This scans an input image for runs (sequences) of 3 or more identical values and replaces them with three such characters followed by a string representing a count of the number of characters. One such implementation of this approach codes the run count into two uppercase hex characters representing a value between 0 and 255 e.g.

A line from a piece of ASCII Art image as follows:

```
,,,,,,]F                                                  8,
```

which is composed of 6 commas, a closed square bracket, a capital F, 48 spaces an eight and a comma, would be represented as:

```
,,,06]F   308,
```

Write a program assignment2_1.c to accept as input either the letter "C", "E" or “R”, followed by a newline. Each subsequent line of input data until EOF contains image data.

If the initial command character was "C", this represents "compress", and the following lines of input text represent an ASCII Art image. Process this input to compress it using the rules specified above and output the compressed version.

If the initial command character was "E", this represents "expand", and the following lines of input text represent a compressed ASCII Art image. Process this input and expand it using the rules referred to above. Print out this uncompressed ASCII Art image.

If the initial command character was "R" this represents "report”. The following lines of input text represent an uncompressed ASCII Art image. Process this input as though you were compressing it, but do not output the image. Instead, at the end of processing, output the efficiency of the system if this image were to be compressed – i.e. the amount by which the original image data is reduced when compressed, expressed as an integer percentage. E.g. if the original image was 200 bytes big and the compressed result was 100 bytes, the efficiency is 50%.

You can test the compression/expansion components of your program by compressing an ASCII Art image, saving the output to a file, editing the file to add E as the first line, then feeding this file back into your program to effectively reproduce the original ASCII Art image. If your program is working correctly then the original image and your compressed, then decompressed version should be identical.

You are guaranteed that there are less than 255 repeated characters in a single run.

Input

Either an ASCII art file with a C on the first line (this will be compressed using RLE)

or

A compressed ASCII art file with an E on the first line (this will be expanded).

or

an ASCII art file with an R on the first line (the storage efficiency that would occur if the image was compressed will be output)

Output

There are three possible outputs from the program:

1. After compressing an input file, all runs of three or more of any character should be replaced by three of the character, and a two-character uppercase hex string representing the count.
2. After expanding an input file, all previously compressed sequences should be replaced by the original character runs.
3. If reporting on the space efficiency of the compression, an integer followed immediately by a ”%” sign.

Note

You can only use stdio.h – no other header files / libraries are permitted.

Sample Input 1

```
C
                                  #####
                   #######      #**#!!###
                  #**#!!!!##   #****#!!!!#
                 #****###!!!#  #*****#!!!!#
                 #********#!!!# #******#!!!!#
                 #**********#!###!*!*!*#!!!!!#        --
                 #!*!*!*!*!*!#!#########!!!!#      /_
                 ###########!##!!!!!!!!!!#!!!#     //__
              ###!!!!!!!!!!!#!!!!!!!!!!!!!!#!!!####///  \
   \       ##!#!!!!!!!!!!!#!!!!!!!!!!!!!!!!#!!!!!!!#
   _\    ##!!#!!!!!!!!!!!#!!!!!!!!!!!#####!!!!!!!*#
    \\  ##!!#!!!!!!!!!!!!#!!!!#######     #!!!!!!***#
  ___\\#!!!####################*****       #...!!*****#
 /   \#!!!.#       ***** #     ***        #....*******#
     #*....#        ***   #               #.......*****#
    #**.....##          *****          ##........!!****#
    #!........##       *******########.......#...!!!!!*#
   #!...........#######.*****...............#.#..!!!!**#
  #*.....##.............#..#...............#...#.!!****#
  #*....#.#............#....#............##......!*****#
  #*.......##.......###......###........#.......!!!****#
  #*........#######......!!....########.......!!!!!***#
   #!!!.................!!!!!!!!.............!!!*******#
    #!!!!............!!!!!!!!!!!!!!!!!!!!!!!!!!!!!******#
     #*******!!!!!!!!!!!!!!!!!!!
```

Sample Output 1

```
  20###05
  12###07   06#**#!!###03
  11#**#!!!04##   03#***04#!!!04#
  10#***04###03!!!03#  #***05#!!!04#
  10#***07#!!!03# #***06#!!!04#
  10#***09#!###03!*!*!*#!!!05#   08--
  10#!*!*!*!*!*!#!###0A!!!04#   06/_
  10###0B!##!!!0A#!!!03#   05//__
  0D###03!!!0B#!!!0D#!!!03###04///03  \
  03\   07##!#!!!0B#!!!0F#!!!07#
  03_\   04##!!#!!!0B#!!!0B###06!!!07*#
  04\\  ##!!#!!!0C#!!!04###07   05#!!!06***03#
 ___03\\#!!!03###13***05   07#...03!!***05#
/   03\#!!!03.#   07***05 #   05***03   08#...04***07#
  05#*...04#   08***03   03#   0E#...07***05#
  04#**...05##   0A***05   0A##...08!!***04#
  04#!...08##   07***07###09...06#...03!!!05*#
  03#!...0B###07.***05...0F#.#..!!!04**#
 #*...05##...0D#..#...0F#...03#.!!***04#
 #*...04#.#...0C#...04#...0C##...06!***05#
 #*...07##...07###03...06###03...08#...07!!!03***04#
 #*...09###07...06!!...04###08...07!!!05***03#
  03#!!!03...11!!!08...0D!!!03***07#
  04#!!!04...0C!!!1B***06#
  05#***07!!!1C***03!!!04***05#
  06#***06!!!19***08!!***04#
  07##***05!!!15#***0D###03
  09##***04!!!15###03***06###04
  0B###04!!!18###06!#
  0F#!!!1E*#
  0F#!!!1D***03##
  0E#!!!1D***06#
  0D#!!!1D***07#
  0C#!!!20***05#
  0B#!!!23**#
  0A#!!!27##
  09#!!!29*##
  08#!!!2B***03#
  08#!!!21###04!!!06***04##
  07#!!!1F###03***04##!!!04***06##
  07#!!!1D##**!!***05#!!!03***08#
  07#!!!08#!!!0B#!!!07#***03!!!04***03!!!04***0A#
  06#!!!0A#!!!09#!!!07#***04!!!05*!!!0A***05#
  06#!!!0B#!!!07#!!!07#*!!***03!!!11***03#
  06#!!!0C#!!!06#!!!06#*!!!05*!!!0F***05#
  06#!!!0C#!!!06#!!!05#***03!!!11***08#
  05#!!!0D#!!!06#!!!05#***04!!!0E***0A#
  05#!!!0D#!!!06#!!!05#***05!!!0E***09#
  05#!!!0D#!!!06#!!!05#***03!!!11***08##
  04##!!!0D#!!!06#!!!05#*!!!15***05#!*##
  03#!#!!!08###06!!!06#!!!05#**!!!0A###09!!!04*#!!**##
 #!#!#!!!06#!!!0C#!!!05#***03!!!05###04***06!!!03###07!!**#
#!#!!##!!!04#!!!0B###0C*!!!03#***08!!!0F**#
#!#!!#!#!!#!!!03#!!!07#!!!0F#***0B!!!0F#
#!#!!!03#!#!#!!#!!!07#!!!04#!!!04#!!!05#***0A!!!0F**#
#!!#!!!03#!##!!#!!!06#!!!04#!!!04#!!!06#***0C!!!0A***04#
 ###09 ###0A!!!04#!!!04#!!!06#***0A!!!0C***03#
  16###11***0C!!!0A**#
  26#***0A!!!05###09
  27###0F
```

Sample Input 2

R

```
                                  #####
                   #######      #**#!!###
                  #**#!!!!##   #****#!!!!#
                 #****###!!!#  #*****#!!!!#
                 #********#!!!# #******#!!!!#
                 #**********#!###!*!*!*#!!!!!#        --
                 #!*!*!*!*!*!#!#########!!!!#      /_
                 ###########!##!!!!!!!!!!#!!!#     //__
              ###!!!!!!!!!!!#!!!!!!!!!!!!!!#!!!####///  \
   \       ##!#!!!!!!!!!!!#!!!!!!!!!!!!!!!!#!!!!!!!#
   _\    ##!!#!!!!!!!!!!!#!!!!!!!!!!!#####!!!!!!!*#
    \\  ##!!#!!!!!!!!!!!!#!!!!#######     #!!!!!!***#
  ___\\#!!!####################*****       #...!!*****#
 /   \#!!!.#       ***** #     ***        #....*******#
     #*....#        ***   #               #.......*****#
    #**.....##          *****          ##........!!****#
    #!........##       *******########.......#...!!!!!*#
   #!...........#######.*****...............#.#..!!!!**#
  #*.....##.............#..#...............#...#.!!****#
  #*....#.#............#....#............##......!*****#
  #*.......##.......###......###........#.......!!!****#
  #*........#######......!!....########.......!!!!!***#
   #!!!.................!!!!!!!!.............!!!*******#
    #!!!!............!!!!!!!!!!!!!!!!!!!!!!!!!!!!!******#
     #*******!!!!!!!!!!!!!!!!!!!
```

Sample Output 2

```
36%
```

Task 2

Title: Sorted count of the unique words in a piece of text

Description

Write a C program called assignment2_2.c, that reads in some input text (e.g. a poem), which features a series of lines (each of no more than 100 characters in length) comprising zero or more words (each word is of at most 30 characters in length) in ASCII (which may include some punctuation characters that must be removed). This input text will contain an unknown number of unique words (each of which may appear several times in the input text). Your C program should process that text into a series of individual words (i.e. strings separated from one another by space, comma or full-stop characters), filter any non-alpha characters from the words, and store them in an appropriate data structure in your program.

The output should be an alphabetically sorted series of words (which will be composed of lowercase characters). Each unique word (along with its associated count) should be output. The sorting of the list of words should be achieved by a function of your own (do not use a library or third-party function to sort the words).

Think carefully about the memory efficiency of your code in order to achieve the highest marks for your solution. You should malloc the storage for the individual strings as you need them, storing them into your data structure. Depending on the data structure you use, you may also need to malloc space for it as it grows.

You may use the string library in your solution to this task.

Input

A number of lines of text (each of 100 characters at most, featuring 0 or more words of at most 30 characters each).

Output

An alphabetically sorted list of lowercase words and their frequency counts. Each line of the output should be in the following format: a unique word from the input text, followed by a space, a "=>" string another space, and then an integer count of how many times that word was found in the text.

Sample Input

```
The Jabberwock
Twas brillig, and the slithy toves
Did gyre and gimble in the wabe:
All mimsy were the borogoves,
And the mome raths outgrabe.
Beware the Jabberwock, my son!
The jaws that bite, the claws that catch!
Beware the Jubjub bird, and shun
The frumious Bandersnatch!
He took his vorpal sword in hand;
Long time the manxome foe he sought
So rested he by the Tumtum tree
And stood awhile in thought.
And, as in uffish thought he stood,
The Jabberwock, with eyes of flame,
Came whiffling through the tulgey wood,
And burbled as it came!
One, two! One, two! And through and through
The vorpal blade went snicker-snack!
He left it dead, and with its head
He went galumphing back.
And hast thou slain the Jabberwock?
Come to my arms, my beamish boy!
O frabjous day! Callooh! Callay!
He chortled in his joy.
Twas brillig, and the slithy toves
Did gyre and gimble in the wabe:
All mimsy were the borogoves,
```

```
And the mome raths outgrabe.
```

Sample Output

```
all => 2
and => 14
arms => 1
as => 2
awhile => 1
back => 1
bandersnatch => 1
beamish => 1
beware => 2
bird => 1
bite => 1
blade => 1
borogoves => 2
boy => 1
brillig => 2
burbled => 1
by => 1
callay => 1
callooh => 1
came => 2
catch => 1
chortled => 1
claws => 1
come => 1
day => 1
dead => 1
did => 2
eyes => 1
flame => 1
foe => 1
frabjous => 1
frumious => 1
galumphing => 1
gimble => 2
gyre => 2
hand => 1
hast => 1
he => 7
head => 1
his => 2
in => 6
it => 2
its => 1
jabberwock => 4
jaws => 1
joy => 1
jubjub => 1
left => 1
long => 1
manxome => 1
mimsy => 2
mome => 2
my => 3
o => 1
of => 1
one => 2
outgrabe => 2
raths => 2
rested => 1
shun => 1
slain => 1
slithy => 2
snickersnack => 1
so => 1
son => 1
sought => 1
stood => 2
sword => 1
that => 2
the => 20
thou => 1
thought => 2
through => 3
time => 1
to => 1
```

```
took => 1
toves => 2
tree => 1
tulgey => 1
tumtum => 1
twas => 2
two => 2
uffish => 1
vorpal => 2
wabe => 2
went => 2
were => 2
whiffling => 1
with => 2
wood => 1
```

HINTS

It is easier to compare two strings if they are converted to the same case (uppercase or lowercase) first.

Your program needs to process text until an EOF is encountered (indicating the end of the text). When testing your program yourself (not via CodeGrade), it is difficult to generate an EOF on some platforms. It is simpler to prepare your test input data in a text file, and then pipe that data into your program using "<" e.g. if your program has been compiled into the a.out file then:

```
a.out < sourcetext.txt
```

will send the contents of sourcetext.txt into your program, and when all the contents of that file have been processed, your program will receive an EOF on the input stream.

You might find it easier to develop your program in stages. Start by writing a program to read the input text and write out each word as it retrieves it from the input stream. You might find the string library function `strtok()` useful for this. When you can successfully do this, the next stage is to store each word in a data structure in your program. e.g. each word could be maintained in a C struct that holds both a string and an integer count of the number of occurrences of the string.

Marks will be awarded for good and efficient use of C structures and pointers and for not using unnecessary storage space (i.e. you should malloc memory space in which to store the word/word-count).